Project 2 Distance Vector Routing

Introduction

In Project 1, you implemented neighbor discovery and flooding. In this project, extend your TinyOS node in TOSSIM to discover minimum-hop routes using distance-vector routing and to forward unicast packets along those routes. Ordinary routed packets should involve only nodes along the selected path, rather than being flooded throughout the network.

You will implement a simplified protocol inspired by RIP. Follow the design slides' separation of routing from data forwarding: DV maintains routes; the IP or network-layer module uses them. Here, IP means the course's simplified forwarding service, not a complete IPv4 implementation. The slides illustrate a recommended architecture; other designs with clean interfaces and equivalent behavior are acceptable.

Objectives and Goals

- Reuse Project 1 neighbor discovery to maintain the current set of usable neighbors.
- Exchange distance estimates with immediate neighbors and maintain destination, next-hop, and hop-count information.
- Update routes when paths improve, become worse, or disappear; recover when an alternative path becomes available.
- Implement split horizon with poison reverse and a bounded metric.
- Use the resulting table for multi-hop forwarding and demonstrate correctness in reproducible simulations.

Where to Start

Scope and assumptions

Use hop count: every usable link has cost 1. Node IDs are at most 255; an ID is an address, not a hop count or a statement that 255 nodes will be present. Preserve reserved addresses in the supplied code. Use bidirectional neighbor relationships confirmed by Project 1; test topologies must explicitly provide both radio directions for each intended usable link.

Use a shared constant INFINITY = 16. Finite routes have costs from 0 through 15; paths requiring 16 or more hops are outside the supported range. All nodes must use the same constant. A node always has a route to itself with cost 0.

Read the textbook's distance-vector subsection as well as the course slides. Focus on Bellman-Ford updates, count-to-infinity, and split horizon. Use course node IDs; full IPv4 headers and addressing, UDP encapsulation, and complete RIP formats are not required.

Local routing state

For every known destination, store its selected cost, next hop, and validity. Keep freshness information sufficient to expire stale learned routes. Initially, only the self-route is valid. A confirmed neighbor has a direct route with cost 1. All other destinations are unknown or unreachable until learned.

Start discovery at boot; update routes as information arrives without waiting for a complete topology. Query the neighbor-table interface; neighbor-change events are recommended. Link-quality statistics may determine neighbor activity, but every active link costs 1.

Cache neighbors' advertised costs, or retain selected routes and learn alternatives from later updates. Both designs must handle increases, withdrawals, and expiration. Document your design and static capacities. Never precompute or inject routes from Python or topology files.

Distance vector advertisements

An advertisement contains destination IDs and distances. DV sends and receives control packets directly through the link-layer service; it must not depend on an existing multi-hop IP route. Distinguish DV, neighbor discovery, flooding, and IP traffic using separate AM types or an explicit link-layer type as illustrated in the design slides. Obtain the advertising neighbor's identity from the link-layer source, not an end-to-end source field.

Because poison reverse depends on the recipient, prepare the advertised vector separately for each neighbor and send it by one-hop unicast. A received advertisement is processed locally and is never forwarded or flooded. A single identical broadcast vector is not sufficient for this recipient-specific rule.

Periodically advertise all known finite routes, including the self-route, and any unreachable entries retained for withdrawal. Choose and document an update interval and expiration intervals long enough for a complete advertisement round and ordinary loss. Triggered updates after route changes are recommended but not required. If used, coalesce changes and introduce a short randomized delay; periodic advertisements must still continue.

Pack as many destination–cost entries as fit after all headers; use multiple bounded packets when needed. Preserve the supplied packet-size configuration. A trailing payload array or equivalent bounded serialization may locate variable-length data, but does not allocate storage. Validate lengths and entry counts before access; never silently truncate the vector.

Omission from one packet means "no new information," not withdrawal. Withdraw explicitly through INFINITY or expiration. Track freshness per destination or equivalently; an unrelated entry must not refresh all routes from that neighbor. Complete-vector reassembly and reliable control-message delivery are not required.

Route updates

For an entry advertising cost c to destination D from neighbor N, the candidate cost is INFINITY if c is INFINITY; otherwise it is the smaller of INFINITY and c + 1. Use arithmetic that cannot overflow before applying the bound. Never replace the self-route with a learned route.

Install a finite candidate if no valid route exists or if it improves the selected route. If the entry is from the current next hop, process increases and INFINITY as well as decreases; accepting only improvements is incorrect. Refresh the relevant learned information when a valid update is received. A worse route from an unrelated neighbor must not displace a better valid route.

Document a stable equal-cost rule: keep a valid current next hop or choose deterministically by neighbor ID. Recompute cached alternatives when their costs or validity change.

Requirements

Failure detection and route withdrawal

Neighbor discovery must continue while routing runs. When a neighbor expires, stop using it as a next hop and discard its cached advertisements, if any. Reevaluate every route using that neighbor. Adopt a currently valid alternative if available; otherwise mark the destination unreachable. Learned routes must also expire when their relevant distance-vector information is no longer refreshed, even if the next hop remains a live neighbor.

Keep a newly unreachable destination available for advertisement with cost INFINITY for a documented withdrawal interval covering multiple periodic rounds. It must not remain usable for forwarding. Afterwards, the entry may be reclaimed. Repeated advertisements of INFINITY must not indefinitely restart the same withdrawal interval. If a finite route is learned again, cancel its withdrawal and restore the route.

Transient loss and temporary routing loops may occur while the protocol converges. After a topology change followed by a stable period with successful control exchanges, reachable destinations within the supported metric range must regain usable minimum-hop routes; disconnected destinations must become unreachable. Neither permanent stale routes nor persistent circulation of an individual packet is acceptable.

Split horizon with poison reverse

When preparing an advertisement for neighbor N, advertise INFINITY for every destination whose selected next hop is N; advertise the selected cost for other destinations. Always advertise the self-route as cost 0. This is split horizon with poison reverse: routes that simple split horizon would omit are explicitly advertised as unreachable to that neighbor. You do not need two separate operating modes.

For example, if A reaches D through B at cost 3, A advertises INFINITY for D to B and cost 3 to another neighbor C. The rule applies whenever the route uses B, not only after a failure. If the next hop changes without a cost change, the appropriate poison-reverse advertisements change too.

Poison reverse addresses two-node routing loops; it does not guarantee freedom from longer loops during convergence. The bounded metric limits count-to-infinity, while the packet TTL bounds how long any one packet can circulate.

Forwarding and local delivery

For a unicast packet destined for this node, deliver it locally according to its protocol. Otherwise, look up a valid route and send the packet to the selected next-hop AM address. Preserve the end-to-end source, destination, protocol, sequence number, and payload; the link-layer next-hop address changes at each hop.

Follow the slides' TTL convention: the source initializes a positive TTL and transmits it unchanged. At an intermediate node, drop a packet with TTL at most 1; otherwise decrement TTL once before forwarding. Deliver a packet locally when its destination matches and TTL is positive. Thus TTL H permits at most H link transmissions: A sends 15 to B, and B forwards 14 to C. Never reset TTL in transit.

Apply the Project 1 duplicate cache only to flooding packets. Ordinary routed IP packets must follow the forwarding and local-delivery rules described above and must not be discarded based on the flooding cache.

If there is no valid route, drop the packet and log the reason. Do not fall back to flooding. Ping replies must use routing as well. Keep neighbor discovery and any retained Project 1 flood traffic in their own handling paths: an AM broadcast does not automatically imply multi-hop flooding.

Debugging and routing interface

Use ROUTING_CHANNEL for forwarding and routing-table output. Each successful forwarding submission should identify the current node, original source, final destination, sequence number, next hop, and selected cost. For example:

DEBUG(1): Routing Packet - src: 3, dest: 10, seq: 0, next hop: 2, cost: 3

When routingTableDump is issued to a node, print that node's table using the existing command handler in command.h. Include columns Dest, NextHop, Cost, and State; display unreachable entries as INF with no usable next hop. Keep these field names consistent in your tests. Log route changes and drops when needed to explain a failure.

Use the high level design slides with DV replacing LS: Ping and future transport call IP; IP queries DV for a route and calls the link layer to transmit. DV exchanges one-hop control messages through the link layer and consults neighbor discovery. Flooding remains a separate Project 1 service. Keep a network-layer protocol discriminator so Ping and transport can coexist; an existing equivalent dispatch mechanism is acceptable. No socket implementation is required here.

Required validation

Supply reproducible Python simulation drivers and topology files. Each test must identify the node boot sequence, directed radio links, noise model, commands, simulated event times, and expected outcome. Measure time in simulated time. Allow startup and post-failure convergence before judging final routes; record your waiting intervals and why they are sufficient.

- Line: use bidirectional links 1–2, 2–3, and 3–4. After convergence, node 1 reaches 4 through 2 at cost 3, and node 4 reaches 1 through 3 at cost 3. Show routed ping and reply without data flooding.
- Alternative path: use links 1–2, 2–4, 1–3, 3–5, and 5–4. Initially node 1 reaches 4 through 2 at cost 2. Remove both directions of link 2–4 while keeping node 2 running. After convergence, node 1 reaches 4 through 3 at cost 3. Restore the link and demonstrate return to cost 2.
- Partition: in the line topology, remove both directions of 2–3. After convergence, node 1 cannot reach 4 and must drop a ping rather than flood it. Restore the link and show recovery. This tests withdrawal, not just finding an alternative.
- Poison reverse: log the advertisements A sends to B and C for a destination routed through B. B must receive INFINITY; C must receive the finite cost. Explain how this prevents B from selecting that reverse route.
- Packet capacity: choose a connected topology whose advertised route set requires multiple packets while its diameter stays below INFINITY. Verify that destinations in later packets are learned and reachable. Show that omission from an individual packet does not withdraw unrelated entries.
- Loss and packet lifetime: under a documented lossy noise configuration, show eventual recovery from missed advertisements once exchanges succeed again. Separately inject a routed ping with a TTL smaller than its path length and show that it expires without a reply or indefinite circulation.

A stopped forwarding node may also be used to test neighbor expiration. The link-removal test is still required because a live node can lose a particular link. Do not require ping to fail at the exact instant of failure: stale state and queued packets can persist briefly.

Hints

- Develop incrementally: verify neighbors, converge routes on a line, route pings, handle withdrawal, add poison reverse, and then test recovery, loss, and larger vectors.
- Use statically allocated routing state, queues, and message buffers. State all capacities, check bounds, and handle a full table or queue without writing outside its storage. Node IDs at most 255 do not require a 256-entry vector to fit in a single packet.
- Use a dedicated routing component where practical. Keep event handlers short; defer larger work to tasks when appropriate. Do not block while waiting for neighbors, replies, or send completion.
- Respect split-phase sends. SUCCESS from a send command means the send was accepted, not that the destination received it. Keep the message buffer valid and unchanged until sendDone. On an immediate error such as EBUSY, do not wait for a completion event for that rejected request; queue or defer the work according to your documented policy.
- Copy received data into owned storage before returning the receive buffer, or correctly implement buffer swapping. Do not retain a pointer to a receive buffer that you returned for reuse.
- Separate AM types still share the radio. Schedule DV updates, neighbor probes, and routed traffic so that control traffic does not indefinitely starve data packets. A bounded queue or staged advertisement round is preferable to a burst of unbounded sends.
- Randomize timer startup or update timing. Keep discovery and routing running continuously; explain the overhead and convergence tradeoff. Do not copy the high level design LS slides' complete-topology startup barrier, Dijkstra computation, or LSA flooding into DV.

Deliverables

Submit one ZIP archive electronically through CatCourses by the beginning of lecture on the due date, unless the course staff announces otherwise. Use a name such as Student1-Student2-proj2.zip. No paper copies are required. The syllabus governs slip days, late submission, teamwork, and academic integrity.

- Source code for the complete working Project 1 and Project 2 stack, with the original project directory structure and files needed to build and run it.
- Simulation scripts, topology and noise inputs, and a short README giving build and run instructions, constants and capacities, and commands for each required test. Include concise representative logs or a test-results summary showing expected and observed outcomes.
- A one-page PDF explaining your routing-state design, update and expiration policy, timer choices, packetization, equal-cost rule, and important implementation tradeoffs. Put command instructions and test logs in the README rather than expanding this design page.
- A PDF with concise answers to the discussion questions. The design write-up and answers may be combined in one PDF, with the design portion limited to one page.

Demonstrate the submitted implementation during the lab as specified in the syllabus. Both team members must attend and explain route computation, poison reverse, failure recovery, and their implementation. Be prepared to run additional staff-provided topologies and commands. The demonstration contributes to the implementation scores; a working program alone does not establish individual understanding.

Discussion Questions

1. Compare distance-vector and link-state routing in terms of stored information, control traffic, convergence, and implementation complexity.
2. Give an example explaining why an update from the current next hop must be processed even when it makes the route worse. What goes wrong if your node accepts only smaller costs?
3. Use a small example to distinguish simple split horizon, poison reverse, and route poisoning after a failure. Why can poison reverse still permit loops involving three or more nodes?
4. Does your implementation necessarily produce symmetric forward and reverse paths? Explain how topology, equal-cost choices, and convergence affect the answer.
5. What happens when a DV advertisement is lost or malformed? Explain how your periodic updates, validation, packetization, and expiration policy address these cases.
6. What happens if a node repeatedly advertises and withdraws a route, or advertises a route but never forwards packets? Describe one possible mitigation and its limitations.

Be concise and relate your answers to the implementation you submitted.

Grading Guidelines

Each part is scored from 0 through 4, then weighted: 0 means nothing submitted; 1 means minimal understanding and little working functionality; 2 means some understanding and some working functionality; 3 means good understanding and most functionality working; 4 means excellent understanding and complete working functionality.

- 70% — Distance-vector implementation, including route updates, expiration, withdrawal, multi-hop forwarding, and required validation.
- 20% — Split horizon with poison reverse, including recipient-specific advertisements and an explanation of their behavior.
- 5% — One-page design write-up.
- 5% — Discussion questions.

Correctness, completeness, and each student's ability to explain the implementation determine the scores. Temporary packet loss during convergence is acceptable; persistent stale routes, buffer misuse, data flooding in place of routing, and packets circulating indefinitely are not.

Efficient Routing Optional Extra Credit

Extra credit is optional and is considered after the regular course grade, as described in the syllabus. First complete and demonstrate the required hop-count protocol. Then design a routing metric that improves performance on a lossy topology, such as an additive link cost based on measured delivery quality.

Explain how you measure the link property and propagate the resulting metric. Preserve bounded costs and failure handling; avoid metrics that accidentally favor loops. Compare against the required hop-count baseline using the same topology, traffic, and noise conditions. Report a relevant measured outcome, such as delivered packets, latency, or retransmission overhead. Include a short design description and reproducible test. Source routing and virtual circuits are not required.

Protocol reference: RFC 2453, Sections 3.4.3 and 3.9.2, provides background on poison reverse and route updates. TinyOS TEP 116 describes packet and send/receive interface contracts. These references provide background; the simplified assignment requirements above define what you must implement.

https://datatracker.ietf.org/doc/html/rfc2453
https://github.com/tinyos/tinyos-main/blob/master/doc/txt/tep116.txt